Literature Review: Pre-Publication Ambiguity Detection in Internet-Drafts

Review Scope

Internet protocol specifications are mainly written in natural language and may still contain undefined terms, conflicting constraints, missing cases, unclear references, or multiple plausible interpretations despite the use of normative keywords such as MUST, SHOULD, and MAY [1, 2].

This project asks whether such ambiguity can be detected in an Internet-Draft revision `-NN` using only information available at or before that revision, and whether revision history improves detection over text-only models. The review therefore focuses on three areas: automated ambiguity detection, protocol-specific ambiguity detection, and revision alignment.

1 Ambiguity Detection in Natural-Language Requirements

Requirements-engineering studies motivate the detector families evaluated in this project. Ezzini et al. compare NLP, machine-learning, and contextual language-model approaches for anaphoric ambiguity [3]. Their results show that supervised learning can be highly effective for ambiguity detection, while contextual models are stronger for resolving the intended reference. This supports comparing classical ML and transformers rather than assuming one family is always superior.

Izhar et al. combine ambiguity rules, TF–IDF features, and Random Forest classification [4]. Their reported results include 97% precision, 89% recall, and 92% F1, showing that engineered linguistic features remain useful for ambiguity classification. Bashir et al. evaluate LLM-based ambiguity detection and explanation in industrial requirements [5]; few-shot examples improved classification over zero-shot prompting, while experts also evaluated explanation quality.

These studies justify the planned comparison of heuristic, classical ML, transformer, and LLM approaches. Their reported scores are not directly comparable with this project because their datasets, ambiguity definitions, and document domains differ from Internet-Drafts.

2 Ambiguity Detection in Protocol Specifications

Protocol-specific studies are the closest prior work.

Yen et al. propose SAGE, which uses Combinatory Categorial Grammar, logical forms, domain-specific disambiguation rules, human clarification, and code generation to detect ambiguous or under-specified protocol text [6]. On the ICMP specification, SAGE identified five ambiguity instances and six under-specification instances. After clarification, the generated implementation interoperated with standard Linux tools. This shows that ambiguity can be detected during specification development. However, SAGE depends on manually designed grammar and human intervention and does not use revision history.

Feng et al. propose RFCKG to detect contradictions between RFC requirements [7]. The system extracts entities and rules, constructs a knowledge graph, and checks direct and conditional contradictions. On CoAP RFC 7252, RFCKG produced 21 alerts, of which five were true contradictions and 16 were false positives. RFCKG is relevant to the inconsistency subtype, but its rule-heavy design and false positives limit its use as a general ambiguity detector.

RFCScope is the closest direct baseline [8]. Pawagi et al. derive seven logical-ambiguity subtypes from 273 verified technical errata and build an LLM pipeline consisting of context construction, semantic partitioning, subtype-aware analysis, and validation. On 20 published RFCs, RFCScope identified 31 new logical ambiguities across 14 RFCs. Eight were confirmed by RFC authors and three became officially verified technical errata.

RFCScope demonstrates that LLMs can detect logical ambiguity requiring cross-section and cross-document reasoning. However, it analyses already published RFCs. It does not test whether the same

issue could have been detected in an earlier Internet-Draft using only information available at that time. This is the main temporal distinction addressed by the proposed project.

3 Revision Alignment and Change Interpretation

Revision history can provide useful evidence, but a changed clause is not automatically an ambiguous clause. Changes may result from editorial rewriting, new functionality, terminology changes, reference corrections, or wider technical redesign.

ARXIVEDITS provides sentence alignment across 751 multi-version scientific papers and models changes such as rewriting, insertion, deletion, splitting, merging, and reordering [9]. CASIMIR extends multi-version analysis to 15,646 scientific articles while preserving sentence correspondence and document position [10].

These studies are relevant to the alignment component rather than ambiguity classification. They support separating two tasks in the proposed benchmark: first, determine which clauses correspond across Internet-Draft revisions; second, determine why the clause changed. Revision reason must therefore remain separate from the ambiguity label.

4 Comparison with Prior Work and Research Gap

Table 1 summarises the main relationship between prior work and the proposed project.

Table 1: Comparison of the most relevant prior work.

Study	**Main Result**	**Relevance**	**Boundary Relative to This Project**
SAGE [6]	5 ambiguity and 6 under-specification cases in ICMP.	Shows that protocol ambiguity can be detected during specification development.	No revision-aware benchmark; grammar- and human-dependent.
RFCKG [7]	5 true contradictions from 21 alerts in RFC 7252.	Supports cross-clause inconsistency detection.	Published RFC; narrow contradiction task; high false-positive rate.
RFCScope [8]	31 logical ambiguities across 14 RFCs; 3 officially verified errata.	Closest ambiguity taxonomy and LLM baseline.	Published RFCs rather than historical Internet-Draft revisions.
ARXIVEDITS / CASIMIR [9, 10]	Multi-version sentence-alignment resources.	Support clause alignment and revision analysis.	Not ambiguity datasets and not Internet-Draft data.
Proposed Project	To be evaluated.	Clause-level pre-publication ambiguity detection using revision-aware evidence.	Uses only contemporaneously available information and compares multiple model families.

The numerical results reported by prior studies cannot be directly ranked against the future results of this project because the tasks, datasets, labels, and evaluation settings differ. For example, SAGE evaluates semantic parsing and code generation, RFCKG evaluates contradiction detection, and RFCScope analyses final published RFCs. Therefore, their published scores or finding counts will be used for contextual comparison rather than direct performance ranking.

The main performance comparison will instead be conducted on the proposed Internet-Draft benchmark. Heuristic rules, classical machine learning, a fine-tuned transformer, an LLM-only detector, and the hybrid ML–LLM pipeline will be evaluated using the same labels and document-family and temporal splits. Where feasible, an RFCScope-inspired detector will also be evaluated on the same benchmark.

Known ambiguity cases from SAGE, RFCKG, and RFCScope may additionally be used as external challenge cases. Their recovery rate can indicate whether the proposed detector generalises to independently identified protocol ambiguities, but such cases should not be treated as a full accuracy or F1 benchmark unless both positive and negative examples are completely reviewed.

The main research gap is therefore the absence of a study that combines:

- ambiguity detection at the Internet-Draft stage;
- clause alignment across successive revisions;
- revision-reason labels separated from ambiguity labels;
- prediction using only information available at the evaluated revision;
- leakage-resistant document-family and temporal evaluation; and
- direct comparison of heuristic, ML, transformer, LLM-only, and hybrid approaches.

The project will also test whether contemporaneous revision features improve F2, PR-AUC, and calibration over text-only models, and whether hybrid routing can preserve detection performance while reducing LLM calls, cost, and review workload.

5 Conclusion

Prior work establishes that ambiguity in protocol specifications is real and detectable. SAGE demonstrates pre-publication disambiguation but relies on manual semantic rules. RFCKG shows that contradictions can span multiple requirements but has limited generality. RFCScope provides the strongest existing LLM-based logical-ambiguity detector, but operates after RFC publication. Revision-alignment studies provide methods for tracing text across versions but do not address ambiguity.

The unresolved question is whether ambiguity can be identified earlier, in a historical Internet-Draft revision, without using information from the future. The proposed project addresses this question through a revision-aware, leakage-resistant benchmark and a controlled comparison of conventional and LLM-based detection methods.

References

[1] S. Bradner, "Key words for use in RFCs to indicate requirement levels," RFC 2119, Mar. 1997. doi: 10.17487/RFC2119.

[2] B. Leiba, "Ambiguity of uppercase vs lowercase in RFC 2119 key words," RFC 8174, May 2017. doi: 10.17487/RFC8174.

[3] S. Ezzini, S. Abualhaija, C. Arora, and M. Sabetzadeh, "Automated handling of anaphoric ambiguity in requirements: A multi-solution study," in *Proceedings of ICSE*, 2022, pp. 187–199.

[4] R. Izhar, S. N. Bhatti, and S. A. Alharthi, "Bridging precision and complexity: A novel machine learning approach for ambiguity detection in software requirements," *IEEE Access*, vol. 13, pp. 12014–12031, 2025. doi: 10.1109/ACCESS.2025.3529943.

[5] S. Bashir et al., "Requirements ambiguity detection and explanation with LLMs: An industrial study," in *Proceedings of ICSME*, 2025, pp. 620–631.

[6] J. Yen, T. Lévai, Q. Ye, X. Ren, R. Govindan, and B. Raghavan, "Semi-automated protocol disambiguation and code generation," in *Proceedings of ACM SIGCOMM*, 2021, pp. 272–286. doi: 10.1145/3452296.3472910.

[7] X. Feng, Y. Zhang, M. H. Meng, and S. G. Teo, "Detecting contradictions from CoAP RFC based on knowledge graph," in *Network and System Security*, LNCS, vol. 13787, 2022, pp. 170–189. doi: 10.1007/978-3-031-23020-2_10.

[8] M. Pawagi, L. Shao, H. Lee, Y. Sun, and W. Wang, "RFCScope: Detecting logical ambiguities in Internet protocol specifications," in *Proceedings of ASE*, 2025, pp. 1234–1246. doi: 10.1109/ASE63991.2025.00106.

[9] C. Jiang, W. Xu, and S. Stevens, "ARXIVEDITS: Understanding the human revision process in scientific writing," in *Proceedings of EMNLP*, 2022, pp. 9420–9435.

[10] L. Jourdan, F. Boudin, N. Hernandez, and R. Dufour, "CASIMIR: A corpus of scientific articles enhanced with multiple author-integrated revisions," in *Proceedings of LREC-COLING*, 2024, pp. 2883–2892.